

JDBC + SQL Basics

Interview Preparation Guide

Java + PostgreSQL Edition

Includes: internal working, real-world examples, complete code, Employee CRUD project, 35 interview Q&A, coding problems, AI-style questions, and a mock interview script.

Table of Contents

1. JDBC

1.1 JDBC Architecture

Internal Working

- JDBC (Java Database Connectivity) is an API that lets Java code talk to any relational database through a uniform interface.
- It has two layers: the JDBC API (java.sql, javax.sql — what your code calls) and the JDBC Driver (vendor-specific implementation, e.g., the PostgreSQL driver 'org.postgresql.Driver').
- Your code calls DriverManager -> gets a Connection -> creates a Statement/PreparedStatement -> executes SQL -> gets back a ResultSet.
- Because the API is standard, switching from PostgreSQL to MySQL mostly means swapping the driver JAR and connection URL — the rest of your code barely changes.
- Under the hood, the driver translates JDBC calls into the database's native wire protocol (e.g., PostgreSQL's frontend/backend protocol).

Real-World Example

Think of JDBC like a universal power adapter: your appliance (Java code) doesn't change, but a different adapter (driver) lets it plug into a different country's socket (database).

1.2 DriverManager

Internal Working

- DriverManager is a factory class that manages a list of registered JDBC drivers and produces Connection objects.
- Since JDBC 4.0, calling Class.forName() is usually unnecessary — drivers self-register via the ServiceLoader mechanism (META-INF/services/java.sql.Driver in the driver JAR).
- DriverManager.getConnection(url, user, password) scans registered drivers and returns the first one whose acceptsURL() matches the given URL.
- For PostgreSQL, the URL format is jdbc:postgresql://host:port/database.

Real-World Example

Similar to how a phone's dialer picks the right carrier/network path based on the number's format — DriverManager picks the right driver based on the URL prefix.

Code Example

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class DriverManagerDemo {
    public static void main(String[] args) {
        String url = "jdbc:postgresql://localhost:5432/company_db";
        String user = "postgres";
```

```

        String password = "postgres";

        try (Connection conn = DriverManager.getConnection(url, user,
password)) {
            System.out.println("Connected: " + !conn.isClosed());
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}

```

1.3 Connection

Internal Working

- A Connection represents a live session with the database — it holds the socket, transaction state, and auto-commit mode.
- Connections are expensive to create (network handshake + authentication), so real applications reuse them via a connection pool (e.g., HikariCP) rather than opening one per query.
- Always close connections (or use try-with-resources) to release database-side resources; leaked connections exhaust the pool.
- Connection also exposes `setAutoCommit()`, `commit()`, and `rollback()` for transaction control.

Real-World Example

Like a phone call to a bank's call center: dialing in (authenticating) takes time, so a busy call center keeps a pool of open lines ready rather than hanging up and redialing for every question.

Code Example

```

try (Connection conn = DriverManager.getConnection(url, user, password)) {
    System.out.println("Auto-commit: " + conn.getAutoCommit());
    System.out.println("Catalog: " + conn.getCatalog());
} catch (SQLException e) {
    e.printStackTrace();
}

```

1.4 Statement

Internal Working

- Statement executes static SQL strings with no parameters — the SQL is sent to the database as plain text.
- `executeQuery()` is used for SELECT (returns a `ResultSet`); `executeUpdate()` is used for INSERT/UPDATE/DELETE (returns affected row count).
- Because the SQL string is built by concatenation, Statement is vulnerable to SQL injection if any part of the query comes from user input — this is the main reason `PreparedStatement` is preferred in real applications.
- Statement is still useful for fixed, parameter-free DDL like `CREATE TABLE`.

Real-World Example

Like handing a fixed, pre-printed form to a clerk — fine when nothing on the form changes, risky if you let a stranger scribble extra instructions onto it.

Code Example

```
import java.sql.*;

try (Connection conn = DriverManager.getConnection(url, user, password);
     Statement stmt = conn.createStatement()) {

    stmt.execute("CREATE TABLE IF NOT EXISTS employee (" +
                 "id SERIAL PRIMARY KEY, name VARCHAR(100), salary NUMERIC)");

    ResultSet rs = stmt.executeQuery("SELECT * FROM employee");
    while (rs.next()) {
        System.out.println(rs.getInt("id") + " " + rs.getString("name"));
    }
} catch (SQLException e) {
    e.printStackTrace();
}
```

1.5 PreparedStatement

Internal Working

- PreparedStatement is a precompiled SQL statement with placeholders (?) for parameters, set via `setInt()`, `setString()`, etc.
- The database parses and plans the query once; subsequent executions with different parameter values reuse that plan — faster for repeated queries.
- Parameters are sent separately from the SQL text (not concatenated as strings), so user input can never be interpreted as SQL code — this is what prevents SQL injection.
- Always prefer PreparedStatement over Statement whenever any value in the query comes from outside the program.

Real-World Example

Like a mail-merge template letter with blanks ('Dear __,') — the letter's structure is fixed and compiled once; only the name in the blank changes per recipient, and the recipient's name can never rewrite the letter itself.

Code Example

```
String sql = "INSERT INTO employee (name, salary) VALUES (?, ?)";
try (Connection conn = DriverManager.getConnection(url, user, password);
     PreparedStatement ps = conn.prepareStatement(sql)) {

    ps.setString(1, "Asha Rao");
    ps.setDouble(2, 75000.00);
    int rows = ps.executeUpdate();
    System.out.println(rows + " row(s) inserted");
}
```

```
} catch (SQLException e) {  
    e.printStackTrace();  
}
```

1.6 ResultSet

Internal Working

- ResultSet is a cursor over the rows returned by a SELECT query; it starts positioned before the first row.
- `rs.next()` advances the cursor and returns false when there are no more rows — the standard iteration pattern is `while (rs.next()) { ... }`.
- Column values are read with typed getters: `getInt()`, `getString()`, `getDouble()`, `getDate()`, etc., addressed by column name or 1-based index.
- By default, ResultSets are forward-only and read-only; scrollable/updatable result sets can be requested via overloaded `createStatement()/prepareStatement()` arguments but are rarely needed.
- A ResultSet is tied to its Statement/Connection — closing the connection invalidates it, which is another reason try-with-resources matters.

Real-World Example

Like scrolling through a spreadsheet one row at a time with a cursor that can only move downward, reading each column of the current row before moving to the next.

Code Example

```
String sql = "SELECT id, name, salary FROM employee WHERE salary > ?";  
try (Connection conn = DriverManager.getConnection(url, user, password);  
    PreparedStatement ps = conn.prepareStatement(sql)) {  
  
    ps.setDouble(1, 50000);  
    try (ResultSet rs = ps.executeQuery()) {  
        while (rs.next()) {  
            int id = rs.getInt("id");  
            String name = rs.getString("name");  
            double salary = rs.getDouble("salary");  
            System.out.printf("%d | %s | %.2f%n", id, name, salary);  
        }  
    }  
} catch (SQLException e) {  
    e.printStackTrace();  
}
```

1.7 CRUD with JDBC

Internal Working

- CRUD = Create, Read, Update, Delete — the four basic data operations, mapped to INSERT, SELECT, UPDATE, DELETE in SQL.

- In JDBC, all four are typically done with PreparedStatement to stay injection-safe and get query-plan reuse.
- executeUpdate() is used for Create/Update/Delete (returns affected row count); executeQuery() is used for Read (returns a ResultSet).
- Well-structured JDBC code wraps CRUD operations behind a DAO (Data Access Object) class so the rest of the application never writes raw SQL.

Real-World Example

A DAO class is like a bank teller window: the rest of the bank (application) never touches the vault (database) directly — it always goes through the teller's fixed set of operations (deposit, withdraw, check balance, close account).

1.8 Transactions

Internal Working

- A transaction groups multiple SQL statements so they succeed or fail as a single unit — this is the 'atomicity' in ACID.
- By default, JDBC connections are in auto-commit mode: every statement commits immediately on its own. For multi-statement transactions, call conn.setAutoCommit(false) first.
- After running the statements, call conn.commit() to make changes permanent, or conn.rollback() to undo everything since the last commit (e.g., inside a catch block).
- Classic use case: transferring money between two accounts — debit one row and credit another; if the credit fails, the debit must be rolled back too, or money disappears.
- Always restore setAutoCommit(true) or close the connection afterward so the pool doesn't hand out a connection stuck mid-transaction.

Real-World Example

Like an ATM cash transfer: money is deducted from your account and added to another's as one indivisible operation — if the power fails halfway, the bank rolls the whole transfer back rather than leaving money 'lost' in between.

Code Example

```
String debit = "UPDATE account SET balance = balance - ? WHERE id = ?";
String credit = "UPDATE account SET balance = balance + ? WHERE id = ?";

try (Connection conn = DriverManager.getConnection(url, user, password)) {
    conn.setAutoCommit(false);
    try (PreparedStatement debitStmt = conn.prepareStatement(debit);
        PreparedStatement creditStmt = conn.prepareStatement(credit)) {

        debitStmt.setDouble(1, 500);
        debitStmt.setInt(2, 101);
        debitStmt.executeUpdate();

        creditStmt.setDouble(1, 500);
        creditStmt.setInt(2, 102);
```

```
        creditStmt.executeUpdate();

        conn.commit();
        System.out.println("Transfer successful");
    } catch (SQLException e) {
        conn.rollback();
        System.out.println("Transfer failed, rolled back");
    } finally {
        conn.setAutoCommit(true);
    }
} catch (SQLException e) {
    e.printStackTrace();
}
```

2. SQL Basics

2.1 SELECT

Internal Working

- Retrieves rows from one or more tables. `SELECT column1, column2 FROM table`; `SELECT *` fetches all columns but should be avoided in production code (fragile to schema changes, wastes bandwidth).
- PostgreSQL executes `SELECT` by scanning the table (or an index, if the planner chooses one) and returns a result set of matching rows.

Real-World Example

Pulling a filtered list of employees to display on an HR dashboard.

Code Example

```
SELECT id, name, salary FROM employee;
```

2.2 INSERT

Internal Working

- Adds a new row to a table. `INSERT INTO table (col1, col2) VALUES (val1, val2);`
- PostgreSQL supports `INSERT ... RETURNING` to get back generated values (like an auto-incremented id) in the same statement — handy in JDBC to avoid a second query.

Real-World Example

Registering a new employee record when HR onboards a hire.

Code Example

```
INSERT INTO employee (name, salary) VALUES ('Rahul Mehta', 62000)
RETURNING id;
```

2.3 UPDATE

Internal Working

- Modifies existing rows. UPDATE table SET col = value WHERE condition;
- Omitting the WHERE clause updates every row in the table — one of the most common and costly SQL mistakes.

Real-World Example

Giving a salary raise to a specific employee after an annual review.

Code Example

```
UPDATE employee SET salary = salary * 1.10 WHERE id = 101;
```

2.4 DELETE

Internal Working

- Removes rows matching a condition. DELETE FROM table WHERE condition;
- Like UPDATE, a missing WHERE clause deletes every row. Many teams require DELETE statements to go through code review or use soft deletes (an is_deleted flag) instead.

Real-World Example

Removing an employee record after they leave the company.

Code Example

```
DELETE FROM employee WHERE id = 101;
```

2.5 WHERE

Internal Working

- Filters rows before they're returned or modified, using comparison operators (=, >, <, <>), logical operators (AND, OR, NOT), and pattern/range operators (LIKE, IN, BETWEEN).
- WHERE is evaluated per row during the scan, so an indexed column in the WHERE clause can let PostgreSQL skip most of the table entirely.

Real-World Example

Filtering an employee table to only those in the 'Engineering' department earning above a threshold.

Code Example

```
SELECT * FROM employee  
WHERE department = 'Engineering' AND salary > 60000;
```

2.6 ORDER BY

Internal Working

- Sorts the result set by one or more columns, ascending (ASC, default) or descending (DESC).
- Sorting happens after filtering (WHERE) but the SQL clause is written last, reflecting SQL's declarative (not execution) order.

Real-World Example

Showing employees ranked from highest to lowest salary on a payroll report.

Code Example

```
SELECT name, salary FROM employee  
ORDER BY salary DESC;
```

2.7 GROUP BY

Internal Working

- Groups rows sharing the same value(s) in specified columns so aggregate functions (COUNT, SUM, AVG, MAX, MIN) can be applied per group instead of over the whole table.
- Every column in the SELECT list that isn't inside an aggregate function must appear in GROUP BY, or PostgreSQL raises an error.

Real-World Example

Calculating total salary expense per department for a budgeting report.

Code Example

```
SELECT department, COUNT(*) AS headcount, SUM(salary) AS total_cost  
FROM employee  
GROUP BY department;
```

2.8 HAVING

Internal Working

- Filters groups produced by GROUP BY, based on aggregate conditions — the group-level equivalent of WHERE.
- WHERE filters rows before grouping; HAVING filters groups after aggregation. You cannot use an aggregate function in WHERE, which is exactly why HAVING exists.

Real-World Example

Finding only the departments whose total salary expense exceeds a budget cap.

Code Example

```
SELECT department, SUM(salary) AS total_cost  
FROM employee
```

```
GROUP BY department  
HAVING SUM(salary) > 300000;
```

3. Coding: Employee CRUD Project

A complete, runnable console application demonstrating full CRUD against PostgreSQL using a DAO pattern and PreparedStatement throughout. Assumes a table created as: CREATE TABLE employee (id SERIAL PRIMARY KEY, name VARCHAR(100) NOT NULL, salary NUMERIC(10,2), department VARCHAR(50));

3.1 Employee.java (Model)

```
public class Employee {  
    private int id;  
    private String name;  
    private double salary;  
    private String department;  
  
    public Employee() {}  
  
    public Employee(int id, String name, double salary, String department) {  
        this.id = id;  
        this.name = name;  
        this.salary = salary;  
        this.department = department;  
    }  
  
    // Getters and setters  
    public int getId() { return id; }  
    public void setId(int id) { this.id = id; }  
    public String getName() { return name; }  
    public void setName(String name) { this.name = name; }  
    public double getSalary() { return salary; }  
    public void setSalary(double salary) { this.salary = salary; }  
    public String getDepartment() { return department; }  
    public void setDepartment(String department) { this.department =  
department; }  
  
    @Override  
    public String toString() {  
        return id + " | " + name + " | " + salary + " | " + department;  
    }  
}
```

3.2 EmployeeDAO.java (Data Access Layer)

```
import java.sql.*;  
import java.util.ArrayList;  
import java.util.List;
```

```

public class EmployeeDAO {
    private static final String URL =
"jdbc:postgresql://localhost:5432/company_db";
    private static final String USER = "postgres";
    private static final String PASSWORD = "postgres";

    private Connection getConnection() throws SQLException {
        return DriverManager.getConnection(URL, USER, PASSWORD);
    }

    // CREATE
    public int addEmployee(Employee emp) throws SQLException {
        String sql = "INSERT INTO employee (name, salary, department) VALUES
(?, ?, ?) RETURNING id";
        try (Connection conn = getConnection();
            PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setString(1, emp.getName());
            ps.setDouble(2, emp.getSalary());
            ps.setString(3, emp.getDepartment());
            try (ResultSet rs = ps.executeQuery()) {
                if (rs.next()) return rs.getInt("id");
            }
        }
        return -1;
    }

    // READ (single)
    public Employee getEmployeeById(int id) throws SQLException {
        String sql = "SELECT * FROM employee WHERE id = ?";
        try (Connection conn = getConnection();
            PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setInt(1, id);
            try (ResultSet rs = ps.executeQuery()) {
                if (rs.next()) {
                    return new Employee(rs.getInt("id"), rs.getString("name"),
rs.getString("department"));
                }
            }
        }
        return null;
    }

    // READ (all)
    public List<Employee> getAllEmployees() throws SQLException {
        List<Employee> list = new ArrayList<>();
        String sql = "SELECT * FROM employee ORDER BY id";
        try (Connection conn = getConnection();
            Statement stmt = conn.createStatement();
            ResultSet rs = stmt.executeQuery(sql)) {
            while (rs.next()) {
                list.add(new Employee(rs.getInt("id"), rs.getString("name"),
rs.getDouble("salary"), rs.getString("department")));
            }
        }
    }
}

```

```

        }
    }
    return list;
}

// UPDATE
public boolean updateSalary(int id, double newSalary) throws SQLException
{
    String sql = "UPDATE employee SET salary = ? WHERE id = ?";
    try (Connection conn = getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setDouble(1, newSalary);
        ps.setInt(2, id);
        return ps.executeUpdate() > 0;
    }
}

// DELETE
public boolean deleteEmployee(int id) throws SQLException {
    String sql = "DELETE FROM employee WHERE id = ?";
    try (Connection conn = getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setInt(1, id);
        return ps.executeUpdate() > 0;
    }
}
}

```

3.3 Main.java (Driver Program)

```

import java.sql.SQLException;
import java.util.List;

public class Main {
    public static void main(String[] args) throws SQLException {
        EmployeeDAO dao = new EmployeeDAO();

        // Create
        int newId = dao.addEmployee(new Employee(0, "Priya Nair", 58000,
"Finance"));
        System.out.println("Inserted employee with id: " + newId);

        // Read all
        List<Employee> all = dao.getAllEmployees();
        all.forEach(System.out::println);

        // Update
        dao.updateSalary(newId, 61000);
        System.out.println("After raise: " + dao.getEmployeeById(newId));

        // Delete
        dao.deleteEmployee(newId);
    }
}

```

```
        System.out.println("Deleted: " + (dao.getEmployeeById(newId) ==  
null));  
    }  
}
```

4. Interview Q&A (35 Questions)

Q1. What is JDBC?

A: JDBC (Java Database Connectivity) is a Java API that provides a standard way for Java applications to connect to and execute SQL against relational databases, regardless of vendor.

Q2. What are the main components of JDBC?

A: DriverManager, Connection, Statement/PreparedStatement/CallableStatement, and ResultSet, plus supporting classes like SQLException and DataSource.

Q3. What is the difference between Statement and PreparedStatement?

A: Statement executes static SQL built by string concatenation and is compiled fresh each time; PreparedStatement is precompiled with placeholders, is faster for repeated execution, and is immune to SQL injection since parameters are sent separately from the SQL text.

Q4. What is SQL injection and how does JDBC prevent it?

A: SQL injection is when untrusted input is concatenated into a SQL string and interpreted as executable SQL. PreparedStatement prevents it because parameter values are bound separately from the compiled query and are never parsed as SQL syntax.

Q5. What is a CallableStatement used for?

A: It's used to call stored procedures/functions in the database, supporting IN, OUT, and INOUT parameters via methods like registerOutParameter().

Q6. What does DriverManager.getConnection() do internally?

A: It iterates over all registered JDBC drivers, calls each driver's `acceptsURL()` to find one that matches the given JDBC URL, and delegates connection creation to that driver.

Q7. Do you need Class.forName() with modern JDBC drivers?

A: No — since JDBC 4.0, compliant drivers self-register with DriverManager via the `java.util.ServiceLoader` mechanism as long as the driver JAR is on the classpath.

Q8. What is a JDBC URL for PostgreSQL?

A: `jdbc:postgresql://host:port/database`, e.g., `jdbc:postgresql://localhost:5432/company_db`, optionally with query parameters like `?ssl=true`.

Q9. Why should you close a Connection?

A: Connections hold real database-side resources (memory, a socket, sometimes a process). Not closing them leaks resources and can exhaust the database's max connection limit or a pool's capacity.

Q10. What is try-with-resources and why is it used with JDBC?

A: It's Java syntax that automatically closes any AutoCloseable resource (Connection, Statement, ResultSet) at the end of the block, even if an exception occurs — this avoids manual finally-block cleanup and resource leaks.

Q11. What is connection pooling and why is it needed?

A: A pool of pre-opened connections reused across requests, avoiding the cost of repeatedly opening/closing TCP + authentication handshakes. Common implementations: HikariCP, Apache DBCP.

Q12. What does auto-commit mode mean?

A: By default, each SQL statement is committed to the database immediately after execution. Multi-statement transactions require disabling it via `conn.setAutoCommit(false)`.

Q13. What are ACID properties?

A: Atomicity, Consistency, Isolation, Durability — the guarantees a database transaction provides: it's all-or-nothing, leaves data in a valid state, is isolated from other concurrent transactions, and survives crashes once committed.

Q14. What is the difference between `executeQuery()` and `executeUpdate()`?

A: `executeQuery()` is for SELECT statements and returns a ResultSet; `executeUpdate()` is for INSERT/UPDATE/DELETE/DDDL and returns an int representing the number of rows affected.

Q15. What does `execute()` (without Query/Update) do?

A: It's a general-purpose method that can run any SQL statement and returns a boolean indicating whether the first result is a ResultSet (true) or an update count (false) — useful when the statement type isn't known in advance.

Q16. What is a ResultSet cursor, and which direction can it move by default?

A: It's a pointer to the current row in the result. By default, ResultSets are TYPE_FORWARD_ONLY — the cursor can only move forward via `next()`, not backward.

Q17. How do you get an auto-generated key after an INSERT in JDBC?

A: In PostgreSQL, append RETURNING id to the INSERT and read it via `executeQuery()`; alternatively, use `Statement.RETURN_GENERATED_KEYS` with `getGeneratedKeys()` in a database-agnostic way.

Q18. What is a batch update in JDBC?

A: Grouping multiple statements (via `addBatch()`) and sending them to the database in a single round trip with `executeBatch()`, reducing network overhead for bulk inserts/updates.

Q19. What is the difference between `rollback()` and `rollback(Savepoint)`?

A: `rollback()` undoes the entire transaction back to its start; `rollback(Savepoint)` undoes only back to a named intermediate point set with `conn.setSavepoint()`, keeping earlier changes in the transaction intact.

Q20. What is a DAO pattern and why use it?

A: Data Access Object pattern: a class that encapsulates all database access for a particular entity, exposing simple methods (add, get, update, delete) so the rest of the application never writes raw SQL directly.

Q21. What SQLException information is useful for debugging?

A: getMessage() for the description, getSQLState() for the standardized error code, and getErrorCode() for the vendor-specific code — chained exceptions are accessible via getNextException().

Q22. What is the difference between WHERE and HAVING?

A: WHERE filters individual rows before grouping/aggregation; HAVING filters groups after aggregation and can reference aggregate functions like SUM() or COUNT().

Q23. Can you use an aggregate function in a WHERE clause?

A: No — aggregates aren't computed yet when WHERE is applied. You must use HAVING for conditions on aggregated values.

Q24. What is the difference between DELETE and TRUNCATE?

A: DELETE removes rows one at a time (can use WHERE, is transactional, fires triggers, is slower); TRUNCATE removes all rows at once, resets identity/auto-increment counters, and is typically faster but can't be selectively filtered.

Q25. What does GROUP BY require regarding the SELECT list?

A: Every selected column that is not wrapped in an aggregate function must appear in the GROUP BY clause, or the database raises an error (in PostgreSQL) since the value wouldn't be well-defined per group.

Q26. What is the difference between INNER JOIN and LEFT JOIN?

A: INNER JOIN returns only rows with matches in both tables; LEFT JOIN returns all rows from the left table plus matching rows from the right table, filling unmatched right-side columns with NULL.

Q27. What is a primary key vs a foreign key?

A: A primary key uniquely identifies each row in its own table; a foreign key is a column in one table that references a primary key in another table, enforcing referential integrity.

Q28. What does NUMERIC(10,2) mean in PostgreSQL?

A: A fixed-point decimal with up to 10 total digits, 2 of which are after the decimal point — commonly used for currency to avoid floating-point rounding errors.

Q29. Why avoid SELECT * in production code?

A: It fetches unnecessary columns (wasting bandwidth), breaks if the schema changes column order, and makes it unclear which columns the code actually depends on.

Q30. What is an index and how does it affect WHERE/ORDER BY performance?

A: An index is a separate data structure (commonly a B-tree) that lets the database locate matching rows without scanning the whole table, speeding up WHERE filters and ORDER BY sorts on indexed columns, at the cost of extra storage and slower writes.

Q31. What is the N+1 query problem?

A: Running one query to fetch a list, then one additional query per item to fetch related data — N+1 total queries instead of a single JOIN or batched query, which severely hurts performance at scale.

Q32. What is the difference between VARCHAR and TEXT in PostgreSQL?

A: VARCHAR(n) enforces a maximum length of n characters; TEXT has no length limit. In PostgreSQL both are stored the same way internally, so TEXT is often preferred unless a specific length constraint is meaningful.

Q33. How would you prevent duplicate employee emails at the database level?

A: Add a UNIQUE constraint (or unique index) on the email column so the database itself rejects duplicate inserts, rather than relying solely on application-level checks that can race under concurrency.

Q34. What isolation level issue does a dirty read describe?

A: A dirty read occurs when a transaction reads data written by another transaction that hasn't committed yet — if that other transaction rolls back, the first transaction has read data that never really existed.

Q35. How do you safely update a row's value based on its current value (e.g., increment) under concurrent access?

A: Perform the update in a single SQL statement (e.g., UPDATE table SET count = count + 1 WHERE id = ?) rather than read-then-write in application code, so the database handles it atomically instead of racing two separate round trips.

5. Coding Problems

5.1 Find the second-highest paid employee (SQL)

Internal Working

- Use OFFSET/LIMIT with DISTINCT salary sorted descending to skip the top value.
- Alternative: a subquery selecting MAX(salary) WHERE salary < (SELECT MAX(salary) FROM employee).

Code Example

```
SELECT DISTINCT salary FROM employee
ORDER BY salary DESC
LIMIT 1 OFFSET 1;
```

5.2 Find employees earning more than their department's average (SQL)

Internal Working

- Use a correlated subquery that computes the average salary per department and compares each employee's salary against it.

Code Example

```
SELECT e.name, e.salary, e.department
FROM employee e
WHERE e.salary > (
    SELECT AVG(e2.salary) FROM employee e2
    WHERE e2.department = e.department
);
```

5.3 Java: Fetch and print employees grouped by department using JDBC

Internal Working

- Run a GROUP BY query via JDBC and use a Map<String, List<Employee>> in Java to organize the ResultSet rows by department as they're read.

Code Example

```
String sql = "SELECT id, name, salary, department FROM employee ORDER BY department";
Map<String, List<Employee>> byDept = new LinkedHashMap<>();

try (Connection conn = DriverManager.getConnection(url, user, password);
    PreparedStatement ps = conn.prepareStatement(sql);
    ResultSet rs = ps.executeQuery()) {

    while (rs.next()) {
        Employee emp = new Employee(rs.getInt("id"), rs.getString("name"),
            rs.getDouble("salary"), rs.getString("department"));
        byDept.computeIfAbsent(emp.getDepartment(), k -> new
ArrayList<>()).add(emp);
    }
}
byDept.forEach((dept, list) -> {
    System.out.println(dept + ":");
    list.forEach(e -> System.out.println("  " + e));
});
```

5.4 Java: Bulk-insert employees using batch updates

Internal Working

- Disable auto-commit, add each insert to a batch via addBatch(), execute all at once with executeBatch(), then commit — far fewer round trips than one executeUpdate() per row.

Code Example

```
String sql = "INSERT INTO employee (name, salary, department) VALUES (?, ?, ?)";
try (Connection conn = DriverManager.getConnection(url, user, password);
    PreparedStatement ps = conn.prepareStatement(sql)) {

    conn.setAutoCommit(false);
    List<Employee> newHires = List.of(
        new Employee(0, "Vikram Shah", 55000, "Sales"),
        new Employee(0, "Neha Kapoor", 67000, "Engineering")
    );
    for (Employee e : newHires) {
        ps.setString(1, e.getName());
        ps.setDouble(2, e.getSalary());
        ps.setString(3, e.getDepartment());
        ps.addBatch();
    }
}
```

```
int[] results = ps.executeBatch();
conn.commit();
System.out.println("Inserted " + results.length + " rows");
}
```

6. AI-Style Interview Questions

Open-ended, scenario-based questions in the style an AI interviewer or take-home panel might ask — testing judgment rather than recall.

- 1. Your JDBC application intermittently throws 'connection refused' under load, but works fine with light traffic. Walk through how you'd diagnose whether this is a connection pool sizing issue, a database max_connections limit, or a network problem.
- 2. A teammate writes all database access using Statement with string-concatenated SQL because 'it's simpler to read.' How would you explain the risk, and what would you show them to make the PreparedStatement version equally readable?
- 3. You need to transfer funds between two employee bonus accounts. Describe the transaction boundaries you'd set, what happens if the JVM crashes mid-transfer, and how the database guarantees no money is lost or duplicated.
- 4. Your Employee CRUD app is slow when loading a dashboard that lists all employees along with their department's average salary. How would you rewrite the queries to avoid the N+1 problem, and what would the resulting SQL look like?
- 5. A junior developer asks why GROUP BY complains when they add an extra non-aggregated column to SELECT. Explain the rule and how you'd fix their query.
- 6. How would you design the employee table's schema to prevent two employees from being inserted with the same email, even if two requests hit the server at the exact same millisecond?
- 7. Your application currently opens and closes a new Connection for every request. Under a load test, response times degrade badly after a few hundred concurrent users. Explain why, and what change you'd make first.

7. Mock Interview Script

A sample back-and-forth to rehearse pacing and depth of answers. Read the interviewer lines aloud, pause, and answer before reading the sample response.

Interviewer: Walk me through what happens, end to end, when your Java code calls `DriverManager.getConnection()` with a PostgreSQL URL.

Sample Answer: *DriverManager looks through all JDBC drivers registered on the classpath — the PostgreSQL driver self-registers via the ServiceLoader mechanism when its JAR is loaded. It calls each driver's `acceptsURL()` until one matches the `jdbc:postgresql://` prefix, then that driver opens a TCP socket to the host and port, performs the PostgreSQL authentication handshake, and returns a Connection object wrapping that session.*

Interviewer: Why would you choose PreparedStatement over Statement even for a query with no user input at all?

Sample Answer: *Mainly consistency and future-proofing — if that query is ever called repeatedly, PreparedStatement lets the database reuse the compiled query plan instead of re-parsing the SQL every time. It also means if someone later adds a parameter derived from user input, the safe pattern is already in place instead of being bolted on.*

Interviewer: Suppose two employees try to update the same salary row at the same time. What could go wrong, and how does a transaction help?

Sample Answer: *Without protection, both could read the old value, compute an update independently, and the second write could silently overwrite the first — a lost update. Wrapping the read-then-write in a transaction with an appropriate isolation level, or better, doing the update as a single atomic SQL statement like `salary = salary + ?` in one round trip, avoids the race because the database serializes the operation itself.*

Interviewer: Your GROUP BY query to compute average salary per department is failing with an error about 'e.name must appear in the GROUP BY clause.' What's happening?

Sample Answer: *I likely selected a column like e.name alongside AVG(e.salary) without including e.name in the GROUP BY list. Since employees within the same department can have different names, the database can't determine a single value for name per group, so it can't pick one arbitrarily — I'd either add name to GROUP BY, or remove it from the SELECT list if I only need the aggregate.*

Interviewer: One last one — how would you explain the difference between WHERE and HAVING to someone new to SQL?

Sample Answer: *WHERE filters individual rows before any grouping happens — it can't see aggregate results because they don't exist yet at that point. HAVING runs after GROUP BY has collapsed rows into groups, so it can filter based on aggregate values like SUM or COUNT per group. A simple way to remember it: WHERE picks which rows go into the groups, HAVING picks which groups survive.*

Final Prep Checklist

- Rebuild the Employee CRUD project from memory end-to-end, including the schema.
- Practice writing PreparedStatement code without referencing this guide.
- Re-derive the WHERE vs HAVING and Statement vs PreparedStatement answers out loud.
- Try the coding problems with a plain-text SQL editor before checking your query against this guide.
- Do a timed run of the mock interview script, answering before reading the sample answer.